DOI 10.1007/s10559-026-00874-3
Cybernetics and Systems Analysis, Vol. 62, No. 3, May, 2026

A COMPARATIVE STUDY OF CHUNKING STRATEGIES FOR RETRIEVAL-AUGMENTED GENERATION

D. Yu Yuvzhenko[1†] **and S. H. Stirenko**[1‡]

UDC 004.8

Abstract. *An empirical comparative study of four document segmentation strategies is presented, namely, of fixed windows of 256, 512, and 1024 tokens, and semantic segmentation based on a large language model. Experiments were conducted on long semantically coherent texts from the SQuALITY dataset. The evaluation was performed on 225 question-answer pairs using Precision@5 and Recall@5 (top-5 retrieval metrics), answer quality metrics (Exact Match and token-level F1), and average retrieval latency. The results reveal a clear trade-off between retrieval precision and recall driven by granularity where smaller fragments provide higher precision, whereas larger fragments substantially increase recall and improve answer quality in terms of F1. Within this experimental setting, semantic segmentation demonstrates competitive results, but does not show a consistent advantage over fixed windows of 512–1024 tokens. A reduction in retrieval latency is observed when using larger segments, which can be explained by lower vector-index density. A reproducible evaluation procedure and practical recommendations for selecting a segmentation strategy for efficient RAG systems are provided.*

Keywords: *Retrieval-Augmented Generation, RAG, chunking, semantic search, long-document question answering, chunking strategies.*

INTRODUCTION

Retrieval-augmented generation (RAG) [1] has become a leading paradigm for extending the capabilities of large language models (LLMs) through external knowledge sources, providing improved factual grounding, interpretability, and robustness across a wide range of information-intensive problems [2]. The combination of a retrieval module, which selects relevant document fragments, with a generative model that produces an answer based on these fragments enables RAG systems to overcome the limitations of fixed model parameters and to update knowledge dynamically without retraining. Thanks to these advantages, RAG has proven effective in problems where useful information may be distributed across large semantically coherent or technical texts.

In this article, the term “fragment” (sometimes “segment”) refers to a part of a document obtained as a result of segmentation.

One of the key, yet often underestimated components of RAG systems is segmentation (chunking), i.e., the process of splitting documents into smaller fragments suitable for retrieval and indexing [3]. Segmentation determines the granularity of the retrieval space, the semantic coherence of the retrieved context, and the structure of the input to the generative model. Despite its importance, optimal segmentation strategies remain unstandardized, and quantitative comparisons of different fragment sizes or segmentation methods were only partially addressed in the literature. The vast majority of modern RAG systems relies on fixed token windows by default, while more advanced approaches, such as semantic or discourse-oriented segmentation, remain insufficiently explored in systematic experiments [4, 5].

[1]National Technical University of Ukraine “Igor Sikorsky Kyiv Polytechnic Institute,” Kyiv, Ukraine, [†]*d.yuvzhenko@kpi.ua*; [‡]*s.stirenko@kpi.ua*. Translated from Kibernetyka ta Systemnyi Analiz, No. 3, May–June, 2026, pp. 42–53; DOI 10.34229/KCA2522-9664.26.3.4. Original article submitted January 11, 2026, accepted February 11, 2026.

© The Authors, under exclusive license to Springer Nature Switzerland AG, part of Springer Nature, 2026
1060-0396/26/6203-0385

The lack of a standardized comparison methodology has the following several important consequences:

— the segment size directly affects retrieval precision and recall where smaller segments typically yield higher precision, but cover a smaller portion of the document, while larger segments provide broader coverage, but may include irrelevant information;

— segmentation influences how coherent and sufficient the context will be for the generative model, especially in long semantically connected texts where the sequence and dependencies between parts of the text are important;

— the number and size of fragments significantly affect retrieval latency and overall system efficiency, in particular, in vector search engines used in large-scale commercial projects.

The combination of these factors necessitates a scientifically grounded investigation into how different segmentation strategies affect the behavior of RAG systems.

The motivation for this study is driven by practical challenges in deploying RAG in domains that require reliable handling of long-context information, such as legal analysis, technical documentation, and the processing of semantically coherent texts. In information retrieval problems from large documents, even minor differences in how fragment boundaries are defined can lead to significant changes in the context that is retrieved, and, consequently, in the quality of the generated response. At the same time, practitioners often lack empirical guidelines to help balance retrieval accuracy, answer quality, and computational efficiency. This article addresses this gap by proposing a controlled comparative study of four segmentation approaches and analyzing their impact on key retrieval and generation metrics.

The objective of this article is to empirically evaluate how the segmentation strategy (fixed windows of 256, 512, and 1024 tokens, as well as semantic segmentation) affects retrieval quality, answer quality, and latency in a RAG system for long semantically coherent texts. The main results and contributions consist of a controlled comparison of segmentation strategies under fixed retrieval and generation components, an analysis of the precision–recall trade-off and its impact on answer quality, and the formulation of practical recommendations for selecting a segmentation strategy in applied RAG scenarios.

SCIENTIFIC LITERATURE REVIEW AND PROBLEM STATEMENT

RAG combines information retrieval methods with generative modeling, enabling LLMs to answer questions based on external information sources rather than relying solely on parametric knowledge. Such a hybrid architecture demonstrates improved factual consistency in problems involving open-ended question answering, text summarization, and multi-step reasoning [6]. Previous research on RAG was primarily focused on improving retrieval modules, in particular, dense vector models, such as Contriever or hybrid architectures that combine sparse and dense retrieval, which significantly affect the accuracy of generated responses [5–8]. However, considerably less attention were paid to the structural properties of the document fragments themselves, that is, how segmentation is performed prior to indexing.

Document segmentation is most commonly performed using fixed windows, typically ranging from 256 to 1024 tokens. This approach originates from early NLP practices and is often applied as a “default setting” rather than as a result of targeted empirical optimization. Fixed windows provide predictability and simplicity for indexing. However, they often ignore logical boundaries, the structure of semantically coherent texts, and the semantic integrity of the content [9]. More recent studies have explored adaptive segmentation methods, including semantic segmentation based on vector representations, hierarchical methods, and splitting strategies that account for rhetorical and structural signals. The goal of such approaches is to achieve alignment between the semantic coherence of a fragment and the granularity of retrieval. However, systematic comparisons across different contexts remain limited.

Studies in question answering over long texts (NarrativeQA, Qasper, and BookQA) have demonstrated the sensitivity of retrieval processes to segment size where smaller segments improve retrieval precision, but reduce document coverage, whereas larger segments, conversely, provide broader coverage at the risk of including irrelevant content. Similar trade-offs were also reported in studies on vector databases where index density, vector granularity, and embedding dimensionality influence retrieval latency and result relevance. However, most publications do not treat segmentation as an independent variable. Instead, it is typically intertwined with the architecture of the retrieval module or with query/prompt engineering strategies for LLMs [9–11].

There are the following several significant gaps in the scientific literature:

— the lack of controlled comparisons. Only a small number of publications directly compare fixed segmentation with semantic segmentation under identical retrieval and generation conditions. Due to differing experimental setups, results across studies are difficult to compare;

— insufficient attention to semantically coherent texts. Question-answering (QA) problems over long semantically connected texts require retrieving information distributed across multiple text fragments. The impact of segmentation in such problems may differ substantially from its impact in fact-based QA problems;

— a near-complete lack of analysis of system-level trade-offs. Most studies optimize for relevance or answer accuracy, while rarely considering retrieval latency or index density, i.e., parameters that are critical for RAG systems in real-world deployment;

— the absence of a standardized evaluation procedure. At present, there is no reproducible experimental protocol or toolkit for assessing segmentation strategies. In most implementations, heuristic configurations are used without clear empirical justification.

The above limitations motivate the following research question: how do different segmentation strategies affect retrieval quality, answer accuracy, and the computational efficiency of RAG systems in question-answering problems over long texts? This question can be decomposed into the following three key dimensions:

— retrieval quality, which is measured by Precision@k and Recall@k (the @k suffix means that the metric is computed over the top-k results returned by the retrieval module), and is directly dependent on the number and semantic coherence of the fragments;

— answer correctness determined by the completeness and relevance of the retrieved context and typically evaluated using EM and F1;

— retrieval latency, which depends on the number of vectors in the vector index and the computational cost of similarity search.

To investigate this problem, a controlled experimental environment is required, in which only the segmentation method is varied, while all other components (the retrieval module, the vector database, and the LLM) remain unchanged. This makes it possible to attribute observed differences specifically to segmentation strategies rather than to other factors.

In what follows, we present such an experimental procedure and apply it to compare four segmentation strategies, namely, fixed windows of 256, 512, and 1024 tokens, and semantic segmentation based on LLMs across 225 question–answer examples from the SQuALITY dataset [12].

OBJECTIVE AND RESEARCH PROBLEMS

The objective of this study is to develop a clear and reproducible experimental procedure (as well as a software toolkit for its implementation) to evaluate how different segmentation strategies affect the performance of RAG (retrieval-augmented generation) systems in question-answering problems over long texts. The proposed approach is designed to isolate segmentation as the primary variable of change. This makes it possible to systematically compare retrieval quality, answer accuracy, and computational efficiency across different document segmentation strategies.

To achieve this objective, it is necessary to do the following:

— develop a complete and reproducible RAG evaluation pipeline that covers document preparation, segmentation, vector representation construction, vector indexing, retrieval, and answer generation;

— implement several segmentation strategies, including fixed sizes of 256, 512, and 1024 tokens, as well as semantic segmentation based on LLMs oriented toward the semantic coherence of fragments;

— construct a controlled dataset for long-text question answering based on SQuALITY, including automated generation of question–answer pairs followed by manual verification and the creation of reference (ground-truth) answers;

— define and compute evaluation metrics, including Exact Match (EM), token-level F1, Precision@5, Recall@5, and average retrieval latency;

— perform a comparative analysis of different segmentation strategies examining trade-offs between retrieval granularity, answer accuracy, and system performance;

— provide practical recommendations for selecting segmentation configurations for real-world RAG systems operating on long-context inputs.

METHODS FOR EXPERIMENTAL EVALUATION OF SEGMENTATION STRATEGIES

The following section describes the used datasets, the document segmentation strategies, the RAG system configuration, and the evaluation procedure. The experiment is designed in such a way that segmentation is the only independent variable where all other components (embedding model, vector search engine, and answer generator) are kept fixed across all runs. This makes it possible to interpret differences in results as a direct consequence of the segmentation strategy.

Object and Subject of the Study. The object of the study is a RAG system configured using several document segmentation strategies. The system combines vector search, an embedding model for constructing vector representations, and an LLM that generates answers.

The subject of the study is the behavior of the retrieval–generation pipeline under different segmentation configurations, in particular, how text granularity and segmentation type affect retrieval quality, answer quality (EM and F1), and system-level characteristics, including retrieval latency and vector index density.

Dataset and QA Pair Generation. The experiments were conducted using the SQuALITY dataset, which contains long semantically coherent human-written texts (ranging from approximately 1,000 to over 4,000 words) along with reading comprehension questions. In contrast to purely fact-based QA datasets, SQuALITY often requires integrating context from multiple paragraphs, making it suitable for evaluating the impact of segmentation in long-document processing scenarios.

For standardized evaluation, a fixed set of 225 QA pairs was constructed. The generation procedure was as follows:

— each story was provided to an instruction template (prompt) to generate three reading comprehension questions along with short reference (gold) answers;

— a custom program developed for this study sent requests to GPT-4o-mini [13] and automatically generated QA pairs from the model's responses;

— the results were normalized and manually reviewed, and ambiguous or incomplete examples were discarded;

— the resulting dataset (225 pairs) was used consistently to evaluate all segmentation strategies.

Reference answers were normalized using rules similar to SQuAD [14], namely, lowercasing, removal of punctuation, elimination of articles (a, an, and the), and whitespace normalization. This reduces the impact of superficial wording differences when computing EM and F1. This normalization ensures fairness in the computation of F1 and EM.

Segmentation Strategies. The study compares four segmentation strategies, covering both widely used fixed approaches and semantic segmentation.

Fixed windows are simple to implement, and they are widely used in practice as a baseline segmentation strategy. In this study, fixed non-overlapping windows of 256, 512, and 1024 tokens were used.

The second approach (LLM-based semantic segmentation) is defined not by a fixed number of tokens, but by semantic transitions. For this purpose, a custom program sends a request to the LLM, which suggests segmentation points (between paragraphs, topical blocks, or logical parts of the text) in order to preserve the semantic coherence of fragments and avoid sentence fragmentation. As a result, fragment length is variable and determined by the structure of the text (in this study it ranges from 300 to 800 tokens).

The selected set of strategies makes it possible to compare the impact of granularity and the "naturalness" of segmentation boundaries on retrieval quality, answer quality, and index characteristics.

System Architecture. In all experiments, the RAG pipeline has the following fixed structure:

— text preprocessing;

— segmentation of text into fragments;

— construction of vector representations of fragments;

— indexing in a vector database;

— retrieval of top-k fragments for a question;

— generation of an answer based on the question and the retrieved context.

As the vector database for similarity search, the Azure AI Search service [15] was used. The text-embedding-3-large model [16] was employed to construct vector representations. Answer generation was performed using the GPT-4o-mini LLM via the Azure OpenAI service [13] with the temperature parameter set to 0, producing more deterministic responses and reducing variability in the LLM outputs.

To properly isolate the effect of segmentation, a separate index in the vector database was created for each strategy all with identical search configurations. The only difference between the indices was the number and structure of fragments produced by the corresponding segmentation method. In all experiments, the retrieval module returned the top-5 fragments for each question.

Evaluation Procedure. This procedure is designed to separately measure retrieval quality (whether the relevant context is found), answer quality, and system efficiency (retrieval latency).

The quality of the model's answers was evaluated using standard QA metrics Exact Match (EM) and token-level F1 as follows:

— EM equals one if the normalized model answer matches the normalized reference answer exactly. Otherwise, it equals zero;

— Token-level F1 measures token overlap between the model's answer and the reference, providing partial credit for incomplete but partially correct responses.

Both metrics were computed after SQuAD-style normalization [14].

Retrieval quality was evaluated using Precision@5 and Recall@5 metrics.

Since segmentation changes the structure of the index, retrieval quality was evaluated independently of generation. For each question, the first k results (top-k) were analyzed where the @k suffix indicates that the metric is computed over the first k retrieved fragments. In this article, $k = 5$.

In this article, Precision@5 and Recall@5 are defined at the document level where a fragment is considered "relevant" if it originates from the same SQuALITY story as the reference answer. This formulation is consistent with the fact that each question in the dataset is linked to a specific source document.

The Precision@5 metric was defined as the proportion of fragments from the correct document among the top-5 retrieved results.

Recall@5 was defined as the proportion of fragments from the correct document that are "covered" by the top-5 results (taking into account the total number of fragments of that document for the given segmentation strategy).

Retrieval latency was measured at the Azure AI Search service level as the time between sending a vector query and receiving the top-5 results. Latency was recorded separately for each question and then averaged across all 225 examples.

Since the corpus contains human-written texts, it may include sensitive passages (e.g., descriptions of violent scenes) that fall under Azure OpenAI content filtering policies [17]. In cases where the model returned an empty or partially generated response, the result was not manually filtered and was evaluated as EM = 0 and F1 = 0. This ensures consistent evaluation rules across all segmentation strategies.

Reproducibility Reproduction. Within this study, a software system was implemented that standardizes the construction of vector indices for each segmentation strategy in the vector database, executes experiments for all QA pairs, and computes metrics in a unified format.

The source code, all required data normalized to a standardized format, the obtained results, and instructions for reproducing the experiment are published in a public GitHub repository [18]. To ensure comparability, all runs were executed with identical settings, and segmentation remained the only independent variable.

EXPERIMENTAL RESULTS

We present the quantitative evaluation results of four segmentation strategies, namely, fixed-256, fixed-512, fixed-1024, and semantic segmentation obtained on 225 examples of long-text question answering (based on SQuALITY). The analysis considers answer quality metrics (EM and F1), retrieval quality metrics (Precision@5 and Recall@5), and system performance (retrieval latency). The aggregated results are shown in Table 1 and Figs 1–4.

TABLE 1. Summary Metrics for Different Segmentation Strategies

Segmentation Strategy	EM	F1	Precision@5	Recall@5	Retrieval Latency (ms)
Fixed-1024	0.0578	0.3770	0.6480	0.4220	1802.35
Fixed-512	0.0489	0.3781	0.7058	0.2386	2103.21
Fixed-256	0.0444	0.3594	0.7289	0.1258	1831.65
Semantic	0.0356	0.3507	0.6907	0.2462	1858.94

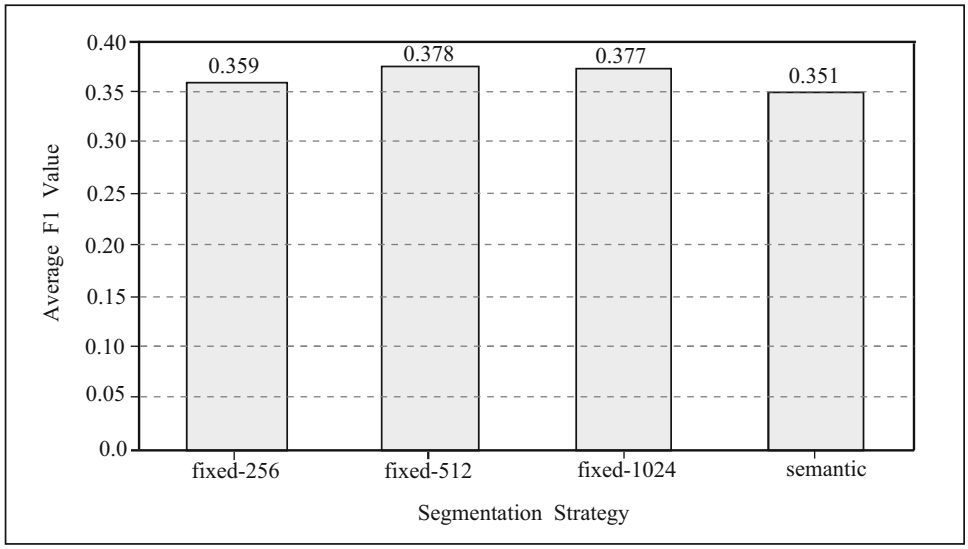


Fig. 1. Average F1 values for each segmentation strategy.

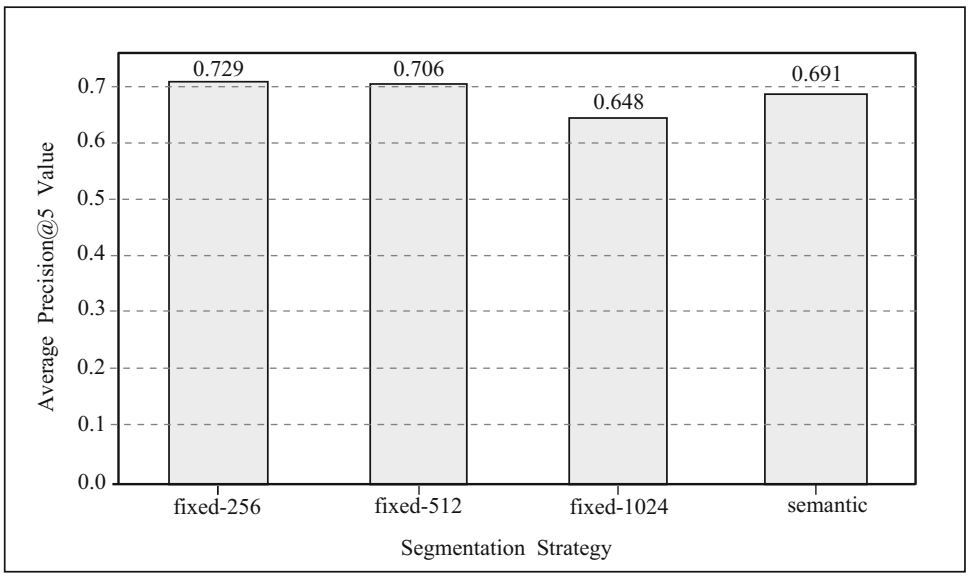


Fig. 2. Average Precision@5 values for each segmentation strategy.

Note that in the retrieval metrics Precision@5 and Recall@5, a fragment is considered "relevant" if it comes from the correct source document (i.e., the same SQuALITY story, to which the reference answer belongs), and the @5 suffix indicates that the metric is computed over the top-5 results.

Answer Quality. Figure 1 shows the average values of F1 for each segmentation strategy.

The highest values of F1 were observed for fixed-512 (0.3781) and fixed-1024 (0.3770). In practice, the difference between them is minimal where fixed-256 shows a lower result (0.3594), while semantic segmentation yields the lowest value (0.3507).

This aligns with intuition since in long texts, especially narrative or semantically coherent ones, the answer is often constructed from multiple fragments, and larger fragments are more likely to provide the generator with a more complete context. With 256 tokens, fragments can be too narrow, and even if the retrieval module operates with sufficient accuracy, the context may be incomplete, which reduces F1. Semantic segmentation in its current implementation typically produces fragments of variable length (within a range close to 512 tokens), but on this dataset it does not yield improvements in F1 compared to fixed windows.

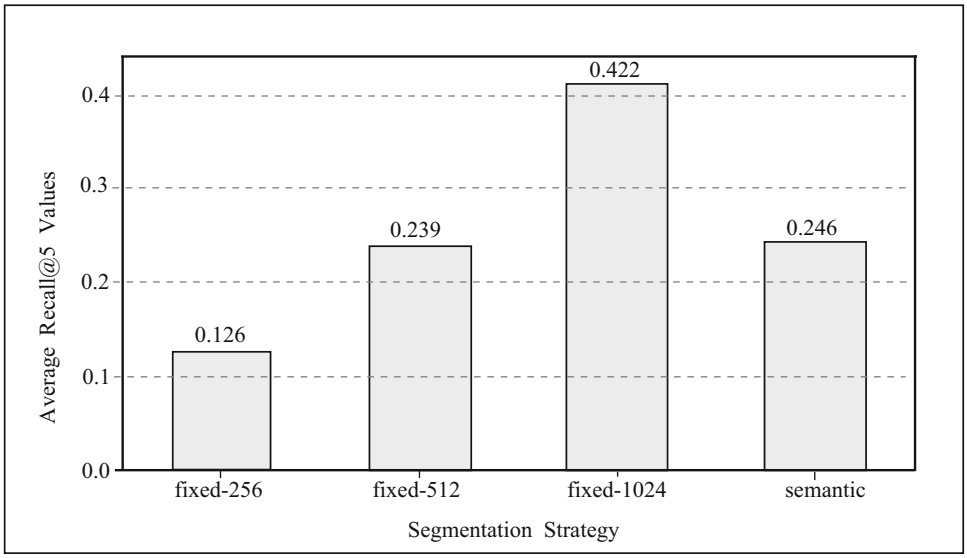


Fig. 3. Average Recall@5 values for each segmentation strategy.

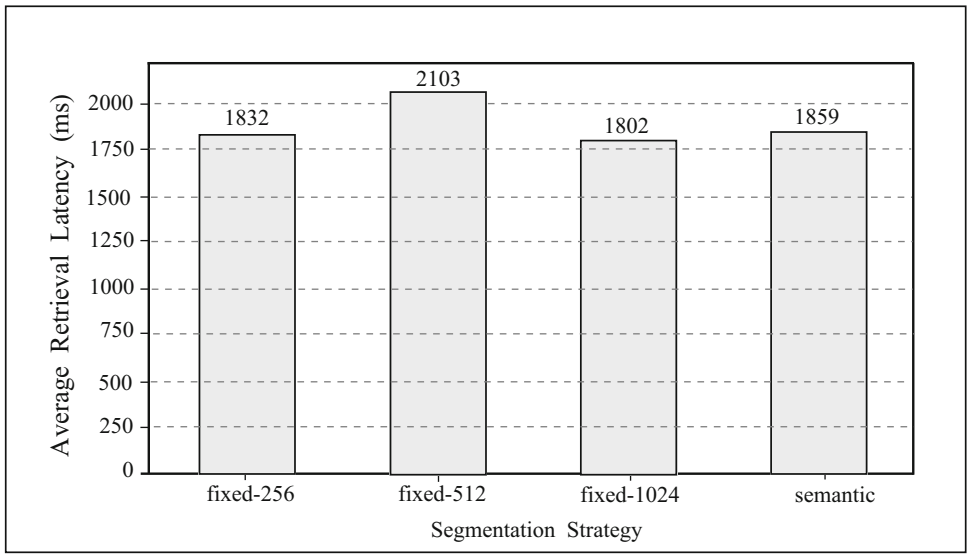


Fig. 4. Average retrieval latency for each segmentation strategy.

The EM values remain low for all strategies (approximately 0.03–0.06), which is generally expected as LLMs often paraphrase answers even when temperature is set to 0, and the reference (gold) answers are short and strictly formulated. For this reason, Fl is a significantly more informative metric of answer quality in this context.

Retrieval Efficiency. The highest Precision@5 value was achieved by the fixed-256 strategy (0.7289). As segment size increases, retrieval precision gradually decreases and for fixed-512 it is 0.7058, and for fixed-1024 it is 0.6480. Semantic segmentation yields an intermediate result (0.6907), close to fixed-512.

Precision@5 shows an inverse correlation with segment size, although the differences are relatively small. The following clear trend can be observed: smaller fragments contain less irrelevant text, which reduces noise within each segment and improves retrieval precision. In contrast, larger segments capture a broader context, but also include additional irrelevant information, which leads to a decrease in precision.

Although semantic segmentation is content-oriented, it generally produces larger segments than fixed-256, which explains its intermediate Precision@5 values.

Recall@5 exhibits the most pronounced contrast between segmentation strategies. The fixed-1024 strategy significantly outperforms all others, reaching a value of 0.422. The fixed-512 and semantic segmentation strategies show intermediate results (approximately 0.24), while fixed-256 demonstrates a very low Recall@5 value (0.126).

This clearly illustrates the trade-off associated with granularity. By definition, Recall@5 depends on how many fragments a document is split into under a given segmentation strategy where if fragments are larger, their number is smaller, and the top-5 results "cover" a larger portion of the source document. For semantically coherent texts where evidence is distributed across multiple paragraphs, such broader coverage often directly improves answer quality (as reflected in F1 for 512–1024 tokens).

Average retrieval latency is shown in Fig. 4. The lowest latency is achieved by fixed-1024 (1802 ms), followed by fixed-256 (1832 ms) and semantic segmentation (1859 ms), while fixed-512 is the slowest (2103 ms).

Overall, latency is consistent with the assumption that it is largely determined by index density since larger fragments result in fewer vectors, which, in turn, speeds up retrieval due to processing fewer vector entries, even though each vector is larger. Deviations from a perfectly monotonic ordering (for example, why fixed-512 is slower than fixed-256) may be attributed to a combination of index density, internal vector database behavior, caching effects, and network overhead in managed cloud infrastructure.

The fastest configuration (fixed-1024) corresponds to the strategy with the smallest number of vectors in the index.

DISCUSSING RESULTS

The experimental evaluation revealed consistent patterns in how segmentation strategy affects retrieval quality, answer quality, and computational efficiency in RAG systems for long-text question answering problems.

The most notable result is the following clear trade-off between retrieval precision and coverage depending on fragment granularity:

— smaller fragments (256 tokens) achieve the highest Precision@5, but have low Recall@5;

— larger fragments (512 and 1024 tokens) reduce Precision@5, but significantly improve Recall@5;

— semantic segmentation occupies an intermediate position.

This trade-off directly carries over to the generation stage. For semantically coherent texts, it is often not enough to merely "retrieve the correct document." The generator also requires a sufficiently rich context containing the necessary details. This is why the fixed-512 and fixed-1024 strategies (with higher recall) achieve the best F1 scores, while fixed-256, despite its high precision, more often provides an overly narrow context, resulting in incomplete answers.

This behavior arises due to fine-grained document segmentation, which produces a significantly larger number of fragments. With a fixed top-k ($k = 5$) setting, the system can physically return only a small portion of the potential context. With larger fragments, the document is represented by fewer segments, so, the top-5 results cover a larger portion of the source text, increasing the likelihood that the necessary evidence is included in the retrieved context.

Let us consider semantic segmentation separately. The hypothesis was that aligning fragment boundaries with semantic transitions should theoretically reduce noise. However, in this study it produces results close to fixed-512 and underperforms compared to fixed-1024 in terms of F1 and Recall@5. A likely explanation is that the typical length of semantic fragments (in the implemented version) does not provide the same level of contextual coverage as 1024-token windows. Additionally, in semantically coherent domains, relevant evidence may be distributed across multiple paragraphs, and "natural" textual boundaries do not always align with the boundaries of the context required to answer the question. LLM-driven segmentation can also introduce variability where, due to suboptimal boundary decisions, important information may sometimes be split across different segments.

Retrieval latency generally aligns with the observation that larger fragments reduce index density (fewer vectors), often speeding up research. At the same time, the non-monotonic ordering of latency between fixed-256 and fixed-512 (with fixed-512 being the slowest configuration) may be explained by a combination of index density and infrastructural overhead in the cloud environment.

It is also worth mentioning an external factor, namely, content filtering in the managed LLM environment. In rare cases, responses may be blocked or truncated, which affects EM and F1. Importantly, the handling rules for such cases are consistent across all strategies in this study, ensuring that comparisons between strategies remain valid.

CONCLUSIONS

This article presents a comparative evaluation of four segmentation strategies (fixed-256/512/1024 and semantic segmentation) in a RAG system for long-form question answering over semantically coherent texts (225 QA examples based on SQuALITY). Segmentation was the only independent variable, while the retrieval and generation components were kept fixed.

Based on the results presented in the article, the following conclusions can be drawn:

— fragment size defines a clear Precision@5–Recall@5 trade-off (smaller fragments are more precise, but provide poorer context coverage, while larger fragments offer significantly better coverage);

— in this domain, answer quality (F1) benefits more from contextual completeness/coverage than from maximal precision of the top-5 retrieved results;

— semantic segmentation is competitive, but within this experiment, it does not demonstrate a consistent advantage over fixed window approaches;

— increasing fragment size generally reduces vector index density, which can lead to lower retrieval latency.

Future work will focus on adaptive and hierarchical segmentation, overlapping segmentation strategies, and position-aware retrieval mechanisms, as well as dynamic context construction to improve retrieval reliability and long-horizon reasoning quality in retrieval-augmented generation systems.

Author Contributions. Both authors have contributed equally to the article. Both authors read and approved the final manuscript.

Funding. The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Conflict of Interest. The authors declare no conflict of interest related to financial, personal, authorial, or other areas that could influence the research and results presented in this article.

REFERENCES

1. P. Lewis, E. Perez, A. Piktus, et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” arXiv:2005.11401 [cs.CL]. https://doi.org/10.48550/arXiv.2005.11401.
2. K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “Retrieval augmented language model pre-training,” in: Proc. of the 37th Intern. Conf. on Machine Learning (ICML 2020) (Virtual Conference, July 13–18, 2020), PMLR, Vol. 119 (2020), pp. 3929–3938. URL: https://proceedings.mlr.press/v119/guu20a.html.
3. R. Qu, F. Bao, and R. Tu, “Is semantic chunking worth the computational cost?” arXiv:2410.13070 [cs.CL]. https://doi.org/10.48550/arXiv.2410.13070.
4. R. Schwaber-Cohen and A. Patel, “Chunking strategies for LLM applications,” Pinecone Systems Inc. (2025). URL: https://www.pinecone.io/learn/chunking-strategies/.
5. O. Khattab and M. Zaharia, “ColBERT: Efficient and effective passage search via contextualized late interaction over BERT,” in: SIGIR’20: Proc. of the 43rd Intern. ACM SIGIR Conf. on Research and Development in Information Retrieval (China, Virtual Event, July 25–30, 2020), Association for Computing Machinery, New York (2020), pp. 39–48. https://doi.org/10.1145/3397271.3401075.
6. D. Yuvzhenko, V. Chymshyr, V. Shymkovych, et al., “A multimodal retrieval-augmented generation system with ReAct agent logic for multi-hop reasoning,” Inf. Comput. and Intell. Syst., No. 6, 42–57 (2025). https://doi.org/10.20535/2786-8729.6.2025.330777.
7. G. Izacard and Grave E., “Leveraging passage retrieval with generative models for open domain question answering,” in: Proc. of the 16th Conf. of the European Chapter of the Association for Computational Linguistics: Main Volume (EACL 2021) (Online, April 19–23, 2021), Association for Computational Linguistics (2021), pp. 874–880. https://doi.org/10.18653/v1/2021.eacl-main.74.

8. V. Karpukhin, B. Oguz, S. Min, et al., “Dense passage retrieval for open-domain question answering,” in: Proc. of the 2020 Conf. on Empirical Methods in Natural Language Processing (EMNLP 2020) (Online, November 2020), Association for Computational Linguistics (2020), pp. 6769–6781. https://doi.org/10.18653/v1/2020.emnlp-main.550.
9. T. Kočiský, J. Schwarz, P. Blunsom, et al., “The NarrativeQA reading comprehension challenge,” TACL, Vol. 6, 317–328 (2018). https://doi.org/10.1162/tacl_a_00023.
10. P. Dasigi, K. Lo, I. Beltagy, et al., “Dataset of information-seeking questions and answers anchored in research papers,” in: Proc. of the 2021 Conf. of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2021) (Online, June 2021), Association for Computational Linguistics (2021), pp. 4599–4610. https://doi.org/10.18653/v1/2021.naacl-main.365.
11. S. Angelidis, L. Frermann, D. Marcheggiani, R. Blanco, and L. Marquez, “Book QA: Stories of challenges and opportunities,” in: Proc. of the 2nd Workshop on Machine Reading for Question Answering (MRQA 2019) (Hong Kong, China, November 2019), Association for Computational Linguistics (2019), pp. 78–85. https://doi.org/10.18653/v1/D19-5811.
12. A. Wang, R. Y. Pang, A. Chen, J. Phang, and S. R. Bowman, “SQuALITY: Building a long-document summarization dataset the hard way,” arXiv:2205.11465 [cs.CL]. https://doi.org/10.48550/arXiv.2205.11465.
13. OpenAI. GPT-4o Mini Model Documentation. URL: https://platform.openai.com/docs/models/gpt-4o-mini (Accessed 25 Dec 2025).
14. P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ questions for machine comprehension of text,” in: Proc. of the 2016 Conf. on Empirical Methods in Natural Language Processing (EMNLP 2016) (Austin, Texas, USA, November 1–5, 2016), Association for Computational Linguistics (2016), pp. 2383–2392. https://doi.org/10.18653/v1/D16-1264.
15. Vector search in Azure AI Search, Microsoft (2024). URL: https://learn.microsoft.com/en-us/azure/search/vector-search-overview.
16. Text-embedding-3-large Model Documentation, OpenAI (2025). URL: https://platform.openai.com/docs/models/text-embedding-3-large.
17. Content filtering overview — Azure OpenAI, Microsoft (2025). URL: https://learn.microsoft.com/en-us/azure/ai-foundry/openai/concepts/content-filter.
18. D. Yuvzhenko, “Comparative study of chunking strategies for retrieval-augmented generation: Experimental code and data repository” (2025). URL: https://github.com/denys-yu/rag-chunk-paper.